

实验报告

学生姓名：	汤承洲	学 号：	20233902058
专 业：	电子信息工程	年级、班级：	23 级 2 班
课程名称：	控制工程	实验项目：	综合练习
实验类型：	☒ 验证 ☐ 设计 ☐ 综合	实验时间：	2026 年 6 月 16 日
实验指导老师：	熊爱民	实验评分：	

1. 实验目的

1. 掌握部分分式展开方法，能使用 MATLAB 的 `residue()` 函数对有理传递函数进行展开，并能推导有源运算放大器网络的传递函数。
2. 掌握控制系统时间域分析方法，能计算系统的单位脉冲响应和单位阶跃响应指标（上升时间、峰值时间、最大超调量、调节时间），并分析增益 K 对系统动态性能的影响。
3. 掌握频域分析方法，能使用 MATLAB 绘制 Bode 图和 Nyquist 图，并读取稳定裕度。
4. 掌握稳定裕度的概念和计算方法，能利用 Bode 图和 Nyquist 图判断系统的稳定性。
5. 掌握 PID 控制器的设计方法，能通过相位裕度分析和极点配置法设计校正环节，改善系统性能。
6. 掌握根轨迹分析方法，能绘制常规根轨迹和参量根轨迹，分析增益和反馈参数对系统闭环极点位置及动态性能的影响。

2. 实验原理

2.1. 部分分式展开与传递函数

部分分式展开是将复杂有理传递函数分解为若干个简单分式之和的方法。

MATLAB 的 `residue()` 函数可计算有理分式的留数、极点和直项：

$$\frac{N(s)}{D(s)} = \sum_{i=1}^n \frac{r_i}{s - p_i} + k(s)$$

其中 r_i 为留数， p_i 为极点， $k(s)$ 为直项（当分子阶次不低于分母阶次时存在）。

运算放大器电路利用“虚短”和“虚断”特性，通过阻抗网络实现各种传递函数。对于反相放大器结构，有 $G(s) = -Z_f(s)/Z_i(s)$ ，其中 Z_f 为反馈阻抗， Z_i 为输入阻抗。

2.2. 时间响应分析

二阶系统的标准传递函数为：

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

其中 ω_n 为自然频率， ζ 为阻尼比。典型阶跃响应指标包括：

- 上升时间 t_r ：响应从终值的 10% 上升到 90% 所需时间
- 峰值时间 t_p ：响应达到第一个峰值所需时间
- 最大超调量 M_p ：峰值超出终值的百分比
- 调节时间 t_s ：响应进入并保持在终值的 $\pm 2\%$ （或 $\pm 5\%$ ）范围内所需时间

增益 K 影响闭环系统的阻尼比： K 增大使阻尼比减小，响应加快但超调增大； K 减小使阻尼比增大，响应变慢但更平稳。

2.3. Bode 图与 Nyquist 图

Bode 图由对数幅频特性曲线和相频特性曲线组成，横轴为角频率的对数刻度，纵轴分别为幅值（dB）和相位（度）。Nyquist 图是在极坐标中绘制频率特性 $G(j\omega)$ 随 ω 从 $0 \rightarrow \infty$ 变化的轨迹，曲线的形状和与临界点 $(-1, j0)$ 的关系可反映系统稳定性。

2.4. 稳定裕度

稳定裕度衡量系统的相对稳定性，包括：

- 幅值裕量 G_m ：系统在相位穿越频率 ω_{cg} 处（相角为 $-\pi$ ），幅值 $|G(j\omega)|$ 的倒数，以 dB 表示时 $G_m(\text{dB}) = -20 \lg |G(j\omega_{cg})|$
- 相角裕量 P_m ：系统在幅值穿越频率 ω_{cp} 处（幅值为 1），相角与 $-\pi$ 的差值

稳定系统要求 $G_m > 1$ ($G_m(\text{dB}) > 0$) 且 $P_m > 0^\circ$ 。

Nyquist 稳定判据：若开环传递函数 $G(s)$ 在右半 s 平面的极点数 $P = 0$ ，则闭环系统稳定的充要条件是 Nyquist 曲线不包围 $(-1, j0)$ 点。

2.5. PID 控制器设计

PID 控制器的传递函数为：

$$G_c(s) = K_p + \frac{K_i}{s} + K_d s$$

其中 K_p 为比例系数， K_i 为积分系数， K_d 为微分系数。

极点配置法通过选择 PID 参数，使闭环系统的极点位于期望位置，从而获得预期的动态性能。对于三阶系统设计三个极点，通过特征多项式系数匹配求解 PID 参数。

2.6. 根轨迹分析

根轨迹是当系统某一参数（通常为开环增益 K ）从 0 变化到 ∞ 时，闭环特征方程的根在 s 平面上的轨迹。绘制依据是幅值条件和相角条件：

$$|G(s)H(s)| = 1, \quad \angle G(s)H(s) = \pm 180^\circ(2k + 1)$$

参量根轨迹用于分析非增益参数（如反馈系数 K_h ）变化时系统极点轨迹，需将特征方程重写为等效开环传递函数形式。

3. 实验内容

3.1. Task 01: 部分分式展开与传递函数（第 2 章）

3.1.1 例题 2-27: 部分分式展开

对传递函数

$$G(s) = \frac{s^4 + 11s^3 + 39s^2 + 52s + 26}{s^4 + 10s^3 + 35s^2 + 50s + 24}$$

进行部分分式展开。使用 `residue()` 函数求得留数 $r = [1, 2.5, -3, 0.5]$ ，极点 $p = [-4, -3, -2, -1]$ ，直项 $k = 1$ 。展开结果为：

$$G(s) = 1 + \frac{1}{s+4} + \frac{2.5}{s+3} - \frac{3}{s+2} + \frac{0.5}{s+1}$$

3.1.2 习题 2-11: 有源电路网络传递函数

推导四个运算放大器电路的传递函数，分别为：

- 电路 (a): 反相放大器， $R_2 \parallel C$ 反馈， $G_a(s) = -\frac{R_2}{R_1(CR_2s + 1)}$ ，为一阶惯性环节
- 电路 (b): 反相放大器， $(R_2 + C) \parallel R_4$ 反馈， $G_b(s) = -\frac{R_4(CR_2s + 1)}{R_1(CR_2s + CR_4s + 1)}$ ，含一阶超前因子
- 电路 (c): T 型网络反馈， $G_c(s) = -\frac{R_2 + R_4 + CR_2R_4s}{R_1}$ ，为一阶比例-微分形式

- 电路 (d): 复杂反馈网络 (两个电容), $G_d(s)$ 为二阶有理分式, 分子含 s 和 s^2 项

3.2. Task 02: 脉冲响应与阶跃响应分析 (第 3 章)

3.2.1 例题 3-5: 单位脉冲响应

系统传递函数为 $G(s) = 50/(25s^2 + 2s + 1)$ 。计算得自然频率 $\omega_n = 0.2000$ rad/s, 阻尼比 $\zeta = 0.2000$ 。单位脉冲响应曲线如图1所示, 最大幅值为 7.5604, 出现在 $t = 6.9078$ s 处。

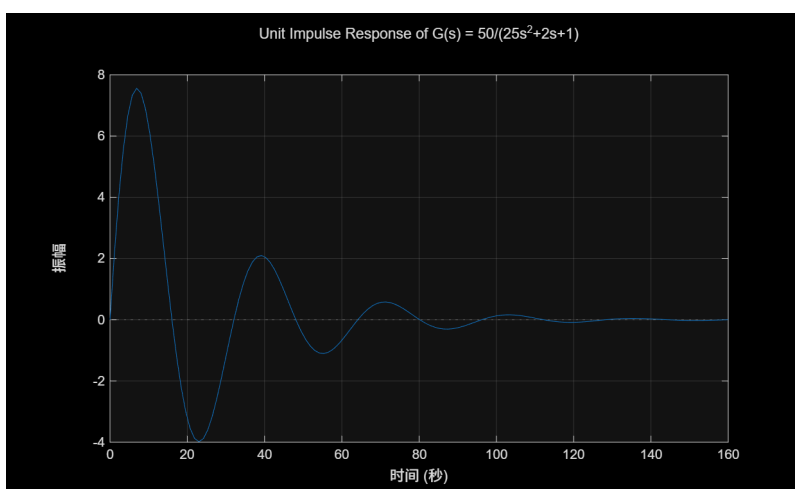


图 1: 单位脉冲响应曲线 (例题 3-5)

3.2.2 习题 3-7: 阶跃响应指标与增益影响

单位反馈系统开环传递函数 $G(s) = 1/(s(s + 1))$, 闭环传递函数 $T(s) = 1/(s^2 + s + 1)$ 。阶跃响应曲线如图2所示, 计算得各项指标:

- 上升时间 $t_r = 1.6390$ s
- 峰值时间 $t_p = 3.5920$ s
- 最大超调量 $M_p = 16.29\%$
- 调节时间 $t_s = 8.0759$ s

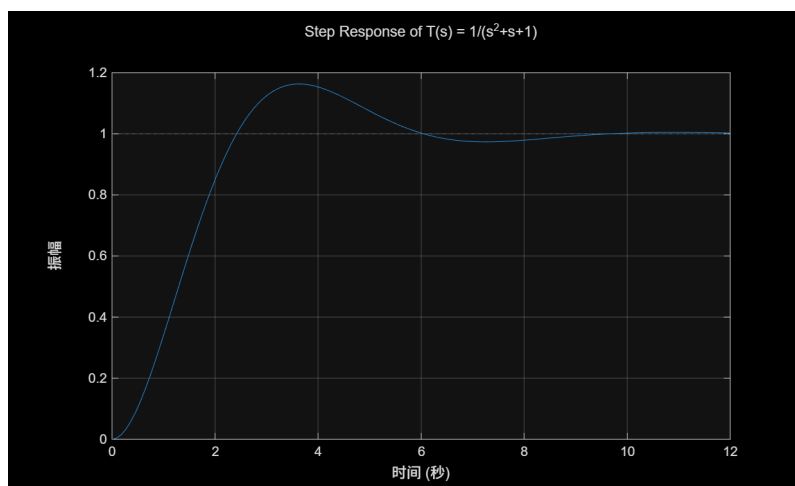


图 2: 阶跃响应曲线 (习题 3-7, $K = 1$)

分析不同增益 K 对系统动态性能的影响, K 分别取 0.5、1、2、5、10, 阶跃响应对比曲线如图3所示, 各项指标汇总于表1。

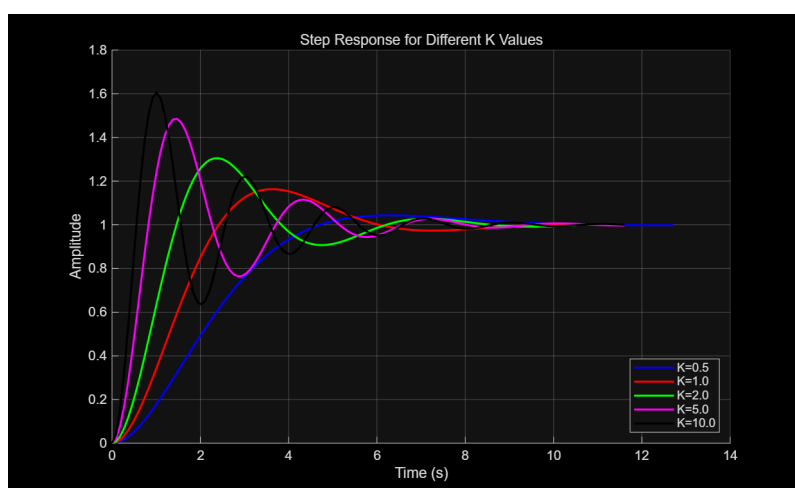


图 3: 不同 K 值下的阶跃响应对比

表 1: 不同增益 K 的阶跃响应指标

K	上升时间 t_r (s)	峰值时间 t_p (s)	超调量 M_p (%)	调节时间 t_s (s)
0.5	3.0390	6.2630	4.32	8.4326
1.0	1.6390	3.5920	16.29	8.0759
2.0	0.9887	2.3947	30.49	7.7420
5.0	0.5533	1.4737	48.52	7.5607
10.0	0.3738	1.0131	60.45	7.3148

分析表明: 随着 K 增大, 系统响应加快 (上升时间从 3.0390s 降至 0.3738s),

但超调量急剧增大（从 4.32% 升至 60.45%），系统由过阻尼（ $K = 0.5$ ）经临界阻尼（ $K = 1$ ）过渡到欠阻尼状态（ $K > 1$ ），振荡加剧。

3.3. Task 03: Bode 图与 Nyquist 图（第 4 章）

3.3.1 例题 4-8: Bode 图

系统 $G(s) = 50/(25s^2 + 2s + 1)$ 的 Bode 图如图4所示。计算得幅值裕量 $G_m = \infty$ （无穷大），相角裕量 $P_m = 3.2725^\circ$ ，增益穿越频率 $\omega_{cp} = 1.4272 \text{ rad/s}$ 。由于系统为二阶，相位永远不会到达 $-\pi$ ，故幅值裕量为无穷大。

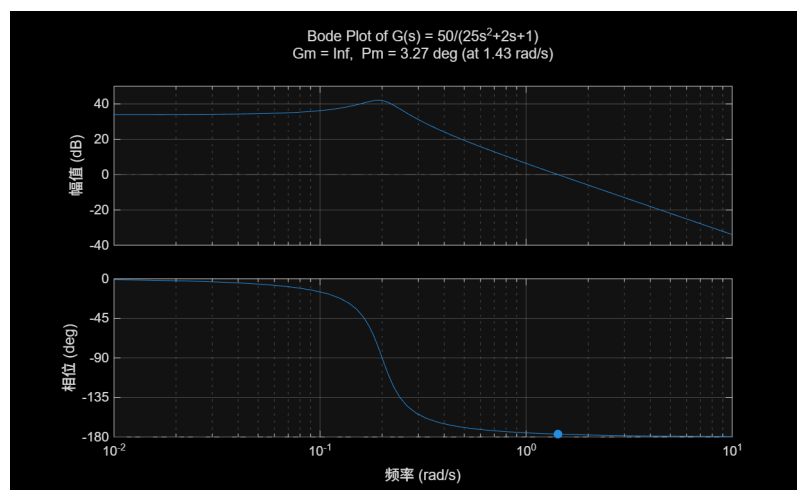


图 4: Bode 图（例题 4-8）

3.3.2 例题 4-10: Nyquist 图

系统的 Nyquist 图如图5所示，局部放大图如图6所示。Nyquist 曲线不包围临界点 $(-1, j0)$ ，系统稳定。

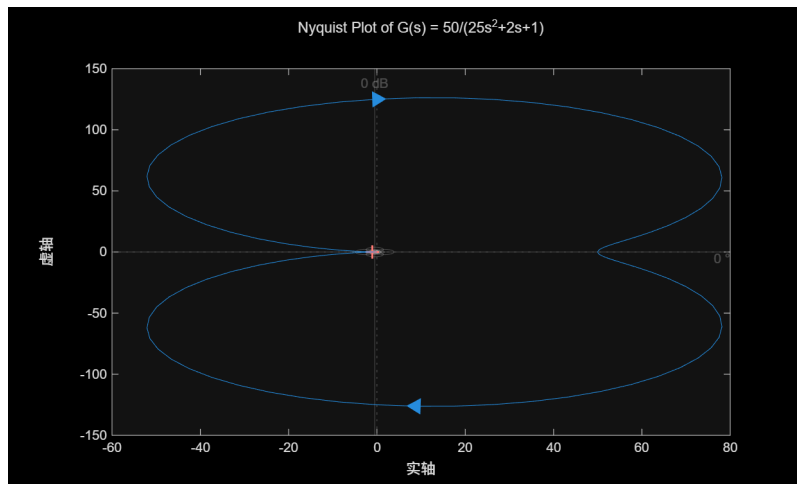


图 5: Nyquist 图 (例题 4-10)

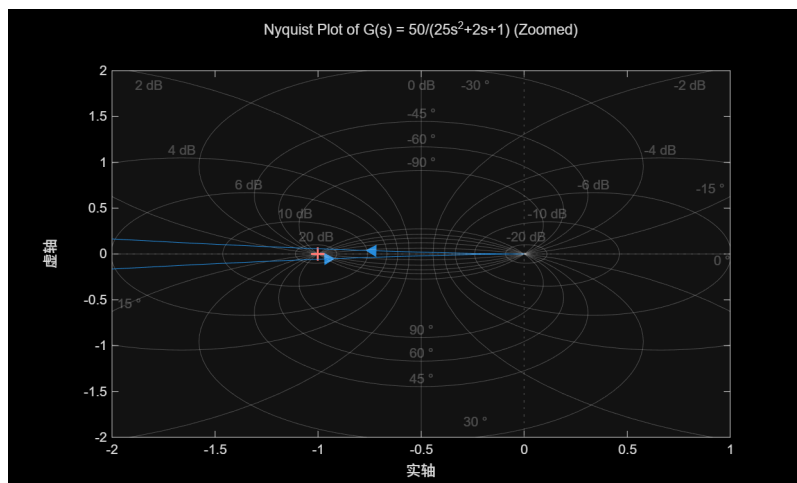


图 6: Nyquist 图局部放大

3.3.3 习题 4-15: Nyquist 图对比

系统 (1) $G(s) = 1/[(s+1)(2s+1)]$ (图??) 和系统 (2) $G(s) = 1/[s^2(s+1)(2s+1)]$ (图??) 的 Nyquist 图对比如图8所示。系统 (1) 为 0 型系统, Nyquist 曲线起始于正实轴; 系统 (2) 含两个积分环节 (II 型系统), 曲线起始于负实轴无穷远处, 在低频段有独特的相位特性。

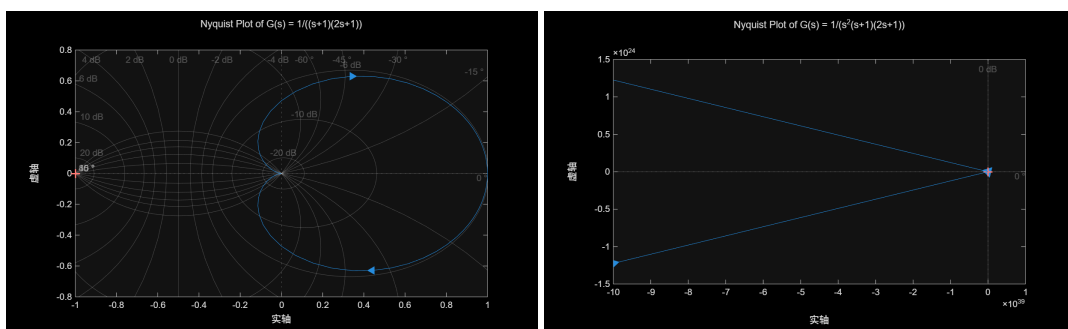


图 7: (左) 系统 (1) Nyquist 图; (右) 系统 (2) Nyquist 图

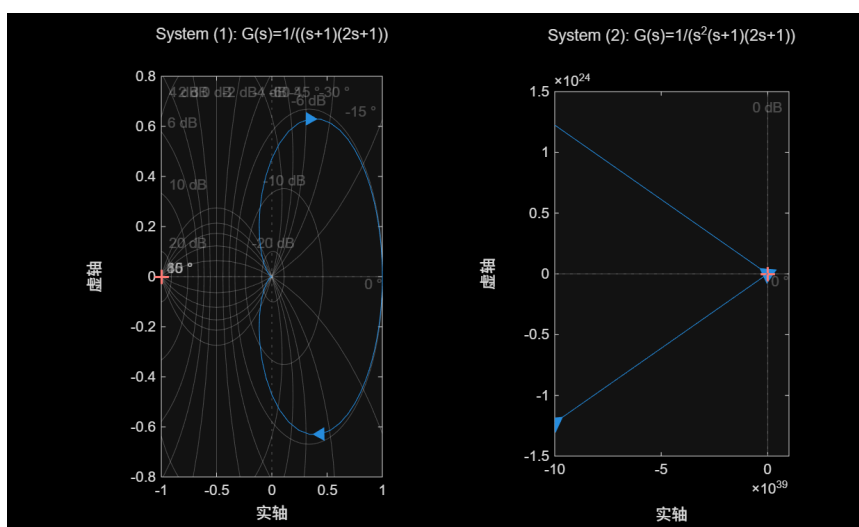


图 8: 系统 (1) 与系统 (2) Nyquist 图对比

3.4. Task 04: 稳定裕度分析（第 5 章）

3.4.1 例题 5-21: 稳定裕度计算

系统 $G(s) = 40/[s(0.1s + 1)(0.05s + 1)]$ 的 Bode 图如图9所示。计算结果:

- 幅值裕量 $G_m = 0.7500$ (-2.50 dB) < 1 , 为负值
- 相角裕量 $P_m = -7.5156^\circ < 0^\circ$, 为负值
- 相位穿越频率 $\omega_{cg} = 14.1421$ rad/s
- 增益穿越频率 $\omega_{cp} = 16.2589$ rad/s

幅值裕量和相角裕量均为负值, 表明闭环系统不稳定。Nyquist 图如图??和图??所示, Nyquist 曲线包围临界点 $(-1, j0)$, 验证了系统的不稳定性。

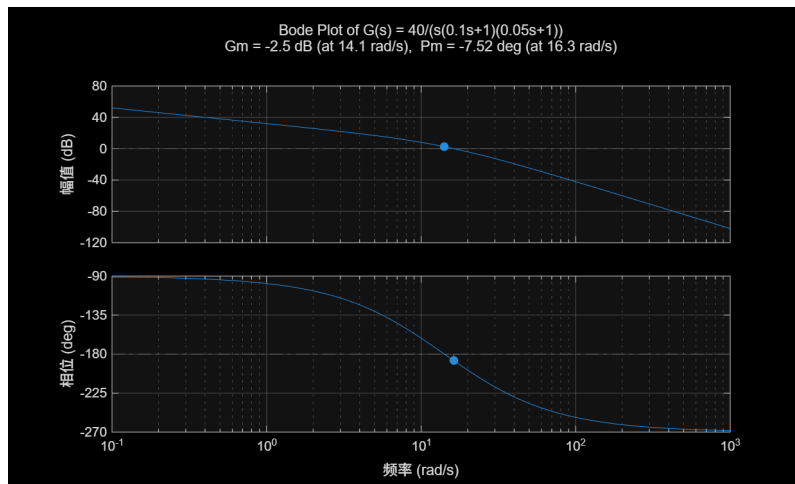


图 9: Bode 图 (例题 5-21)

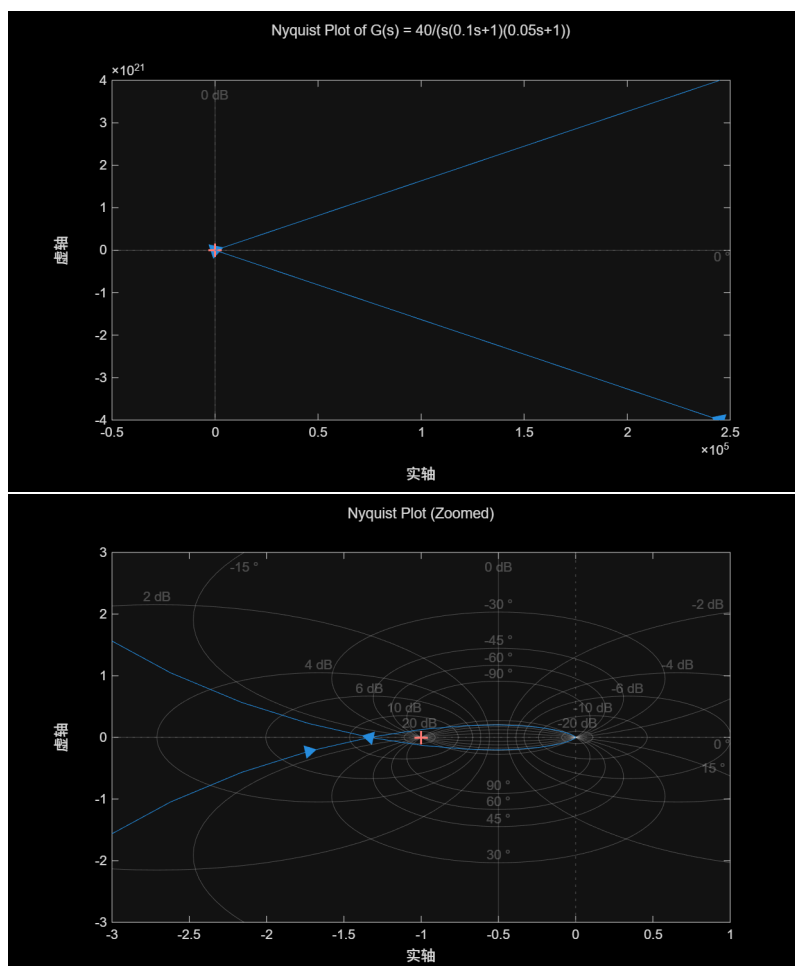


图 10: (上) Nyquist 图; (下) Nyquist 图局部放大

3.4.2 习题 5-22: 稳定性分析

系统 $G(s) = 10/[s(s+1)(s+10)]$ 的 Bode 图如图 11 所示。计算结果:

- 幅值裕量 $G_m = 11.0000$ (20.83 dB) > 1
- 相角裕量 $P_m = 47.4041^\circ > 0^\circ$
- 相位穿越频率 $\omega_{cg} = 3.1623$ rad/s
- 增益穿越频率 $\omega_{cp} = 0.7844$ rad/s

闭环极点为 -10.1086 、 $-0.4457 \pm 0.8892i$ ，全部位于左半 s 平面，系统稳定。Nyquist 图（图??和图??）曲线不包围临界点。阶跃响应（图13）验证了系统的稳定性。

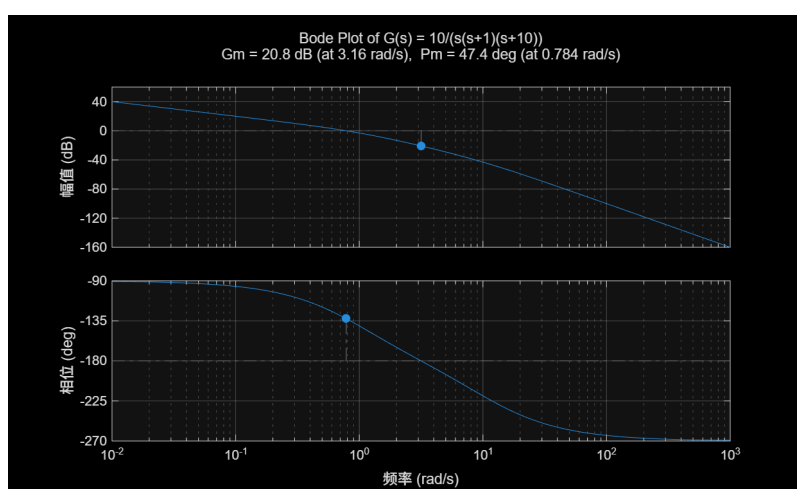


图 11: Bode 图 (习题 5-22)

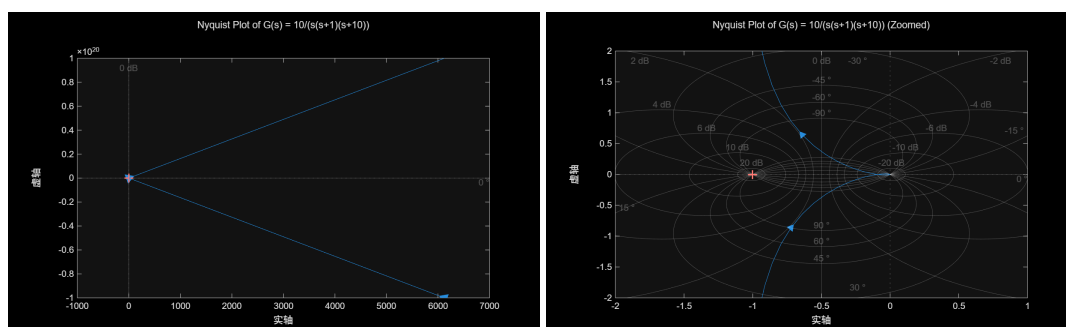


图 12: (左) Nyquist 图; (右) Nyquist 图局部放大

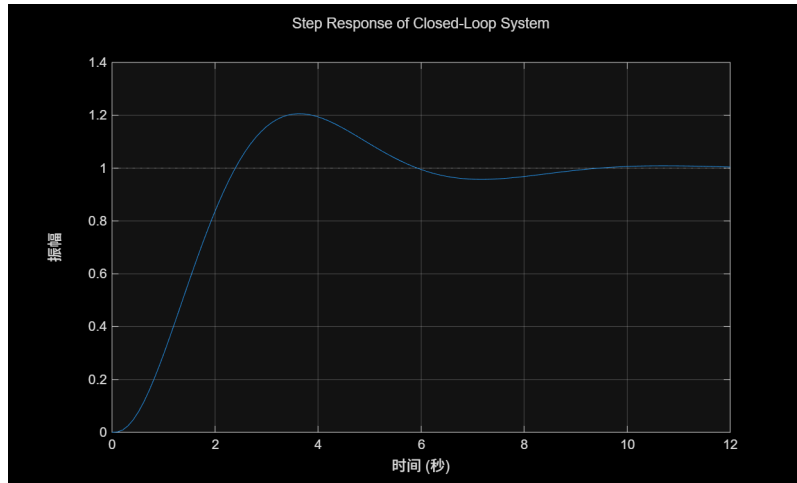


图 13: 闭环系统阶跃响应

3.5. Task 05: PID 控制器设计 (第 7 章)

3.5.1 例题 7-6: 校正前后相位裕度对比

校正前系统 $G_1(s) = 100/[s(0.04s + 1)(0.01s + 1)]$ 的 Bode 图如图??所示:

- 幅值裕量 $G_m = 1.2500$ (1.94 dB)
- 相角裕量 $P_m = 5.2057^\circ$
- 稳定裕度不足, 接近临界稳定

采用滞后-超前校正后:

$$G_2(s) = \frac{100(0.5s + 1)}{s(5s + 1)(0.04s + 1)(0.01s + 1)}$$

Bode 图如图??所示, 稳定裕度显著提升:

- 幅值裕量 $G_m = 11.3734$ (21.12 dB), 提升了约 19.18 dB
- 相角裕量 $P_m = 53.0740^\circ$, 提升了约 47.87°

校正前后的阶跃响应对比如图15所示, 校正后系统的超调量显著减小, 响应更加平稳。

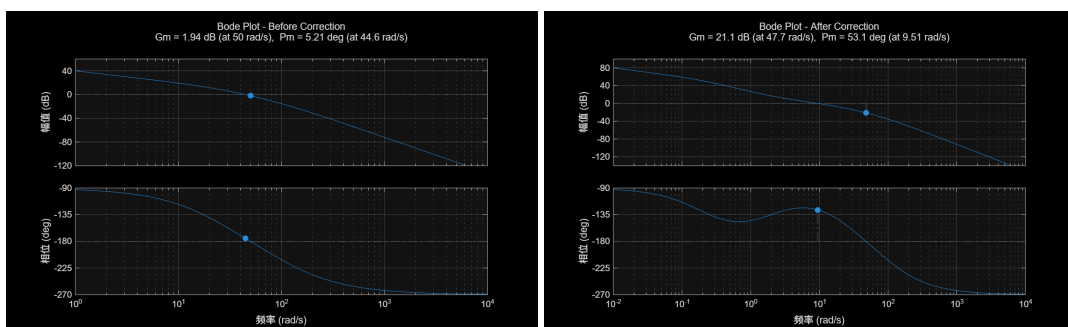


图 14: (左) 校正前 Bode 图; (右) 校正后 Bode 图

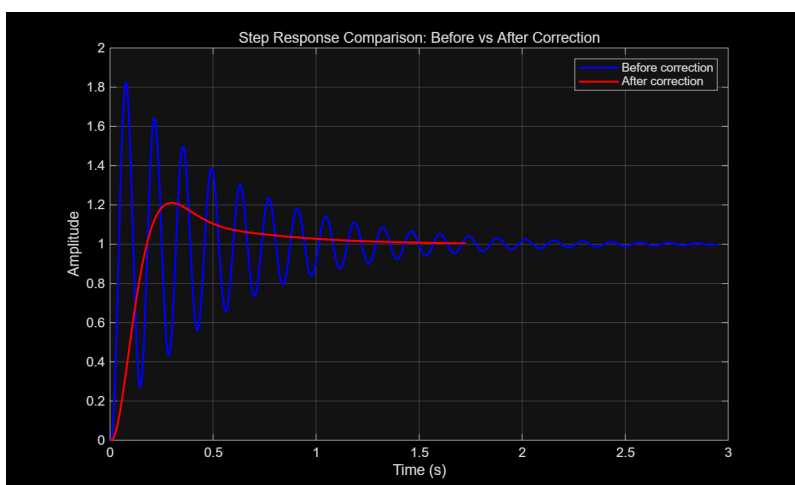


图 15: 校正前后阶跃响应对比

3.5.2 习题 7-20: PID 控制器极点配置

被控对象 $G_0(s) = 100/[s(10s + 1)]$ ，期望闭环极点为 $-2 \pm j1$ 和 -5 。通过系数匹配法设计 PID 控制器：

$$C(s) = K_p + \frac{K_i}{s} + K_d s = 2.5000 + \frac{2.5000}{s} + 0.8900s$$

验证极点配置：实际闭环极点精确为 -5 、 $-2 + j1$ 、 $-2 - j1$ ，与期望完全吻合。闭环阶跃响应（图16）指标：

- 上升时间 $t_r = 0.1527$ s
- 峰值时间 $t_p = 0.4421$ s
- 超调量 $M_p = 19.32\%$

- 调节时间 $t_s = 1.9142 \text{ s}$
- 稳态值为 1（零稳态误差）

补偿后系统的 Bode 图如图17所示。

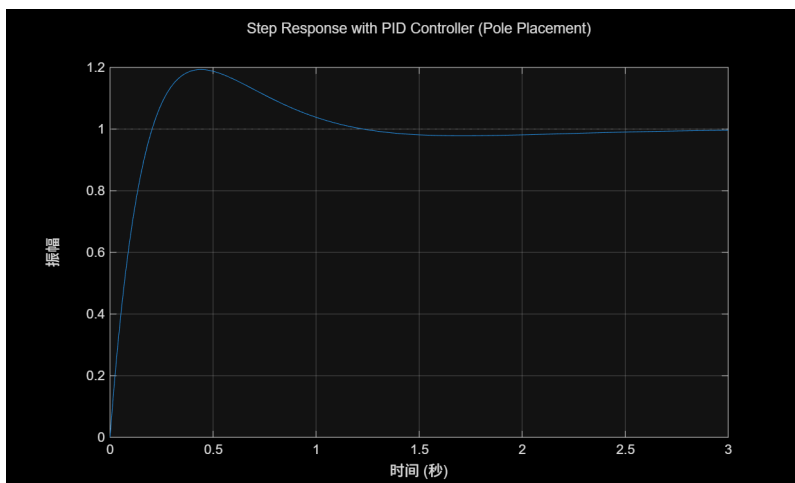


图 16: PID 控制阶跃响应

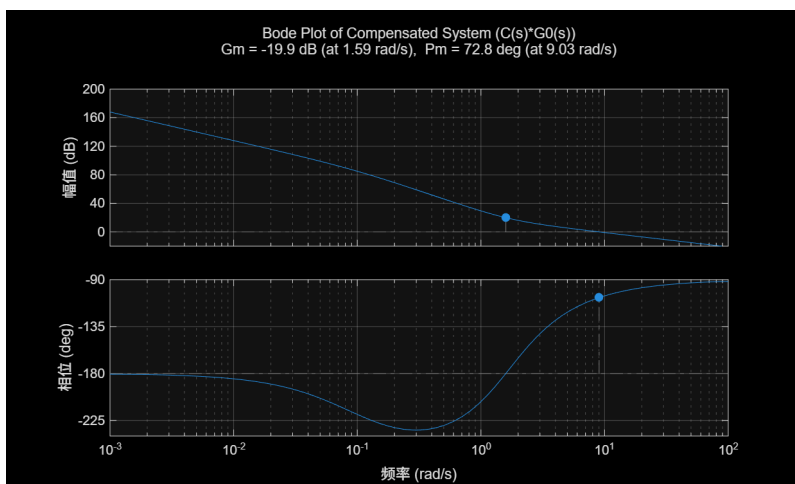


图 17: 补偿后系统 Bode 图

3.6. Task 06: 根轨迹分析（第 8 章）

3.6.1 例题 8-12: 根轨迹

系统 $G(s) = K/[s(s+2)(s+5)]$ 的根轨迹如图18所示。分离点计算：由 $dK/ds = 0$ 得 $-3s^2 - 14s - 10 = 0$ ，分离点 $s = -0.8804$ ，对应增益 $K = 4.0607$ 。测试点 $s = -1+j1$ 处增益 $K = 8.2462$ ，对应闭环极点 -5.4405 、 $-0.7797 \pm 0.9527i$ 。

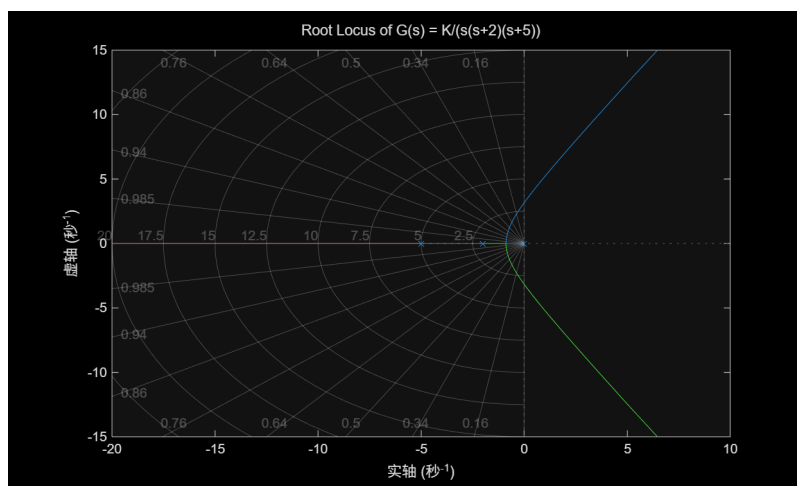


图 18: 根轨迹 (例题 8-12)

3.6.2 习题 8-8: 含速度反馈的根轨迹

系统结构为前向通道 $K/[s(s+1)]$, 反馈通道 $1 + K_h s$ 。

Part (1): $K_h = 0.5$, K 从 $0 \rightarrow \infty$ 的根轨迹如图19所示。等效开环传递函数 $G(s)H(s) = K(0.5s + 1)/[s(s+1)]$ 。

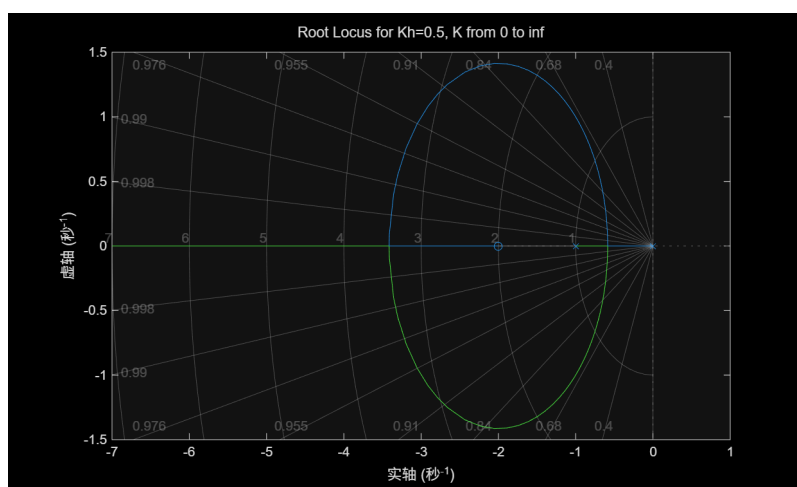


图 19: 根轨迹 ($K_h = 0.5$, K 变化)

Part (2): $K_h = 0.5$, $K = 10$ 时, 闭环极点为 $-3 \pm j1$, 阻尼比 $\zeta = 0.9487$ (高阻尼)。

Part (3): $K = 1$, K_h 从 $0 \rightarrow \infty$ 的参量根轨迹如图20所示。等效开环传递函数 $G_{eq}(s) = s/(s^2 + s + 1)$ 。

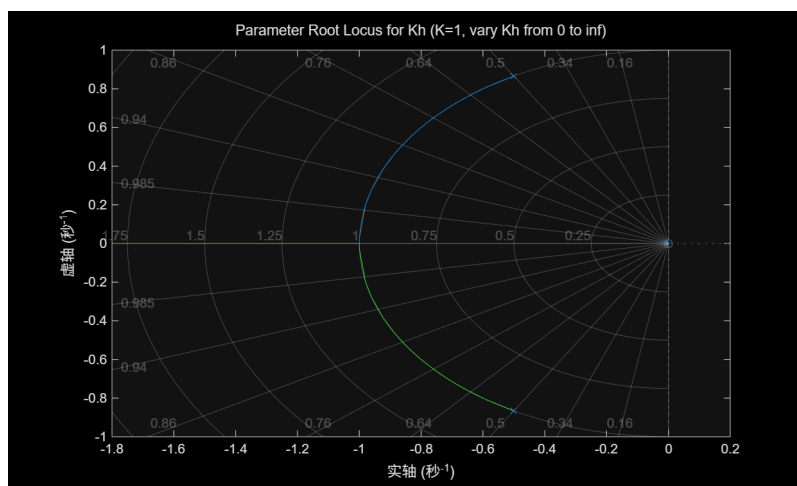


图 20: 参量根轨迹 ($K = 1$, K_h 变化)

Part (4): $K = 1$ 时不同 K_h 下的阶跃响应如图21所示，指标汇总于表2。

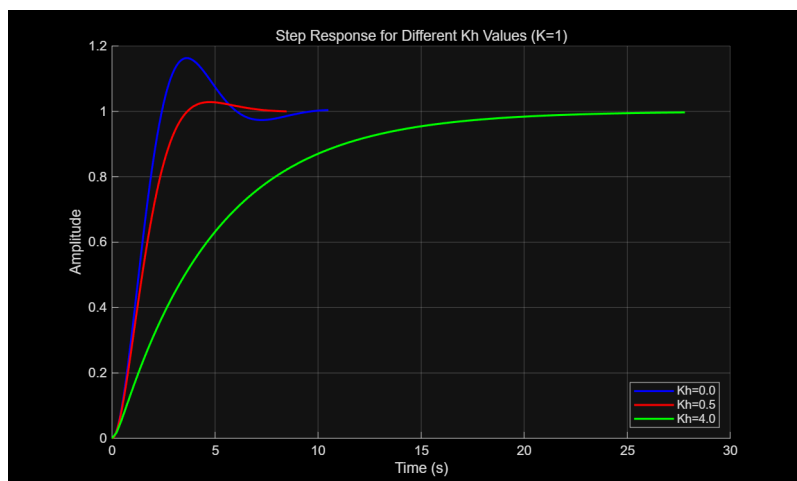


图 21: 不同 K_h 下的阶跃响应

表 2: 不同 K_h 值的阶跃响应指标 ($K = 1$)

K_h	超调量 M_p (%)	调节时间 t_s (s)	峰值时间 t_p (s)	上升时间 t_r (s)
0.0	16.29	8.0759	3.5920	1.6390
0.5	2.84	5.7426	4.7280	2.2884
4.0	0.00	18.9574	35.0829	10.5355

分析表明: K_h 增大使阻尼比显著提高, 超调量从 16.29% ($K_h = 0$, 欠阻尼) 降至 2.84% ($K_h = 0.5$), 最终降为 0% ($K_h = 4.0$, 过阻尼)。但 K_h 过大时调节时间和上升时间显著延长 (t_s 从 8.08s 增至 18.96s, t_r 从 1.64s 增至 10.54s), 表

明速度反馈在抑制超调的同时也减缓了响应速度。结论：适中的 K_h （如 0.5 左右）可在超调和响应速度之间取得较好平衡。

4. 实验总结

本次综合练习实验涵盖了控制工程课程的核心内容，通过 MATLAB 编程解决了教材第 2 章至第 8 章的典型例题和习题，系统性地应用了控制系统的时域、频域分析和设计方法。

1. 实验一（部分分式与传递函数）：掌握了 `residue()` 函数的部分分式展开方法，成功将四阶有理传递函数分解为四个一阶分式和一个直项的和。通过符号推导完成了四种运算放大器电路的传递函数解析式，从简单的一阶惯性环节到含两个电容的复杂二阶网络，深化了对模拟电路传递函数推导的理解。
2. 实验二（脉冲与阶跃响应分析）：计算了二阶系统的自然频率和阻尼比，获取了完整的阶跃响应指标（ t_r 、 t_p 、 M_p 、 t_s ）。系统分析了增益 K 对动态性能的影响规律： K 增大使响应加快但超调增加，系统从过阻尼经临界阻尼过渡到欠阻尼状态，验证了经典控制理论中增益与阻尼比之间的反比关系。
3. 实验三（Bode 图与 Nyquist 图）：熟练使用 `margin()` 和 `nyquist()` 函数进行频域分析。理解了不同类型系统 Nyquist 图的特征差异（0 型系统起始于正实轴，II 型系统起始于负实轴无穷远处），以及 Bode 图中幅值裕量和相角裕量的物理意义。
4. 实验四（稳定裕度分析）：通过两个典型系统对比了稳定与不稳定系统的裕量特征。不稳定系统的幅值裕量和相角裕量均为负值，Nyquist 曲线包围临界点；稳定系统则相反。结合 Nyquist 稳定判据、闭环极点位置和阶跃响应多方面验证了稳定性判断的准确性。

5. 实验五 (PID 控制器设计): 完成了滞后-超前校正设计, 使相角裕量从 5.21° 大幅提升至 53.07° , 稳定裕度显著改善。通过极点配置法精确设计了 PID 控制器参数 ($K_p = 2.5$, $K_i = 2.5$, $K_d = 0.89$), 使闭环极点精确到达期望位置 ($-2 \pm j1$ 和 -5), 实现了良好的动态性能。
6. 实验六 (根轨迹分析): 绘制了常规根轨迹, 计算了分离点和对应增益。通过参量根轨迹分析了速度反馈系数 K_h 对系统动态性能的影响规律: K_h 增大可有效抑制超调 (从 16.29% 降至 0%), 但代价是响应速度变慢 (调节时间从 8.08s 增至 18.96s), 设计中需权衡。

通过本次综合练习, 全面巩固了控制工程的理论知识, 熟练掌握了 MATLAB 控制系统工具箱的使用方法, 为后续更复杂的控制系统分析、设计和仿真任务奠定了坚实基础。

5. 实验源码

5.1. Task 01: 部分分式展开与传递函数 (task1.m)

```
1  %% task1.m - Experiment 7, Task 1: Partial Fraction Expansion and
   Transfer Functions
2  % Example 2-27: Partial fraction expansion using residue()
3  % Exercise 2-11: Derive transfer functions for 4 op-amp circuits
4
5  clc;
6  clear;
7  close all;
8  diary('results/logs/Task01_output.log');
9
10 fprintf('==== Task 1: Partial Fraction & Transfer Function ==== \n');
11 fprintf('\n');
12
13 %% Part 1: Example 2-27 - Partial Fraction Expansion
14 fprintf('--- Part 1: Partial Fraction Expansion (Example 2-27) --- \n');
15 ;
16 fprintf('\n');
17
18 % Define numerator and denominator of G(s)
19 num = [1, 11, 39, 52, 26];
20 den = [1, 10, 35, 50, 24];
```

```

20
21 fprintf('G(s) = (s^4 + 11s^3 + 39s^2 + 52s + 26) / (s^4 + 10s^3 + 35s
    ^2 + 50s + 24)\n');
22 fprintf('\n');
23
24 % Use residue() for partial fraction expansion
25 [r, p, k] = residue(num, den);
26
27 fprintf('Partial Fraction Expansion Results:\n');
28 fprintf('Residues (r):\n');
29 disp(r');
30 fprintf('Poles (p):\n');
31 disp(p');
32 fprintf('Direct term (k):\n');
33 disp(k');
34
35 fprintf('Expanded form:\n');
36 for i = 1:length(r)
37     if imag(p(i)) == 0
38         fprintf(' %.4f / (s + %.4f)\n', r(i), -p(i));
39     else
40         fprintf(' (%.4f %+.4fi) / (s %+.4f %+.4fi)\n', real(r(i)),
            imag(r(i)), -real(p(i)), -imag(p(i)));
41     end
42 end
43 fprintf('\n');
44
45 fprintf('\n');
46
47 %% Part 2: Exercise 2-11 - Op-Amp Circuit Transfer Functions
48 fprintf('--- Part 2: Op-Amp Transfer Functions (Exercise 2-11) ---\n')
    ;
49 fprintf('\n');
50
51 % Define symbolic variables
52 syms R1 R2 R3 R4 R5 C C1 C2 s
53
54 %% Circuit (a): R1 input, R2||C feedback, R3 to ground on non-
    inverting
55 fprintf('Circuit (a): Inverting amplifier with R2||C feedback\n');
56 fprintf(' Input impedance Zi = R1\n');
57 fprintf(' Feedback impedance Zf = R2 || (1/sC)\n');
58
59 % Zf = R2 || (1/sC) = (R2 * (1/sC)) / (R2 + 1/sC) = R2 / (1 + s*R2*C)
60 Zf_a = R2 / (1 + s*R2*C);
61 Zi_a = R1;
62 G_a = -Zf_a / Zi_a;
63 G_a = simplify(G_a);
64

```

```

65 fprintf(' G_a(s) = Vo/Vi = -Zf/Zi = \n');
66 pretty(G_a);
67 fprintf('\n');
68
69 %% Circuit (b): R1 input, (R2 series C) || R4 feedback, R3 to ground
70 fprintf('Circuit (b): Inverting amplifier with (R2+C) || R4 feedback\n
    ');
71 fprintf(' Feedback branch: Z1 = R2 + 1/(s*C)\n');
72 fprintf(' Feedback impedance: Zf = Z1 || R4\n');
73
74 Z1 = R2 + 1/(s*C);
75 Zf_b = (Z1 * R4) / (Z1 + R4);
76 Zi_b = R1;
77 G_b = -Zf_b / Zi_b;
78 G_b = simplify(G_b);
79
80 fprintf(' G_b(s) = Vo/Vi = -Zf/Zi = \n');
81 pretty(G_b);
82 fprintf('\n');
83
84 %% Circuit (c): T-network feedback: R2 to node, C from node to GND, R4
    from node to output
85 fprintf('Circuit (c): Inverting amplifier with T-network feedback\n');
86 fprintf(' Structure: (-)---R2---N---R4---Output, with C from N to GND
    \n');
87 fprintf(' Using nodal analysis at node N and virtual ground at (-)\n'
    );
88
89 syms VN Vo Vi_node
90 eq1 = (0 - VN)/R2 + (0 - VN)*s*C + (Vo - VN)/R4 == 0;
91 eq2 = Vi_node/R1 == (0 - VN)/R2;
92 sol = solve([eq1, eq2], [VN, Vo]);
93 G_c = simplify(sol.Vo / Vi_node);
94
95 fprintf(' G_c(s) = Vo/Vi = \n');
96 pretty(G_c);
97 fprintf('\n');
98
99 %% Circuit (d): Complex feedback network
100 syms VA
101 % Impedances
102 Z2 = R2 + 1/(s*C1); % Series R2 + C1
103 Z4 = R4 + 1/(s*C2); % Series R4 + C2
104
105 % At node A: current from (-) + current through R5 + current to GND =
    0
106 eqA = (0 - VA)/Z2 + (Vo - VA)/R5 + (0 - VA)/Z4 == 0;
107
108 % At (-) node: Vi/R1 = (0 - VA)/Z2

```

```

109 eqIn = Vi_node/R1 == (0 - VA)/Z2;
110
111 sol_d = solve([eqA, eqIn], [VA, Vo]);
112 G_d = simplify(sol_d.Vo / Vi_node);
113
114 fprintf(' G_d(s) = Vo/Vi = \n');
115 pretty(G_d);
116 fprintf('\n');
117
118 %% Display all transfer functions in a clean summary
119 fprintf('\n==== Summary of Transfer Functions ==== \n');
120 fprintf('\n');
121 fprintf('G_a(s) = \n'); disp(G_a);
122 fprintf('G_b(s) = \n'); disp(G_b);
123 fprintf('G_c(s) = \n'); disp(G_c);
124 fprintf('G_d(s) = \n'); disp(G_d);
125
126 fprintf('\nTask 1 completed.\n');
127 diary off;

```

5.2. Task 02: 脉冲响应与阶跃响应分析 (task2.m)

```

1  %% task2.m - Experiment 7, Task 2: Impulse & Step Response Analysis
2  % Example 3-5: Unit impulse response, compute natural frequency and
   % damping ratio
3  % Exercise 3-7: Step response metrics, effect of gain K on dynamics
4
5  clc;
6  clear;
7  close all;
8  diary('results/logs/Task02_output.log');
9
10 fprintf('==== Task 2: Impulse & Step Response Analysis ==== \n');
11 fprintf('\n');
12
13 %% Part 1: Example 3-5 - Unit Impulse Response
14 fprintf('--- Part 1: Unit Impulse Response (Example 3-5) --- \n');
15 fprintf('\n');
16
17 % Define the system:  $G(s) = 50 / (25s^2 + 2s + 1)$ 
18 num1 = [50];
19 den1 = [25, 2, 1];
20 G1 = tf(num1, den1);
21
22 fprintf('System:  $G(s) = 50 / (25s^2 + 2s + 1)$  \n');
23 fprintf('\n');
24
25 % Compute natural frequency and damping ratio

```

```

26 wn = sqrt(1/25);      % natural frequency
27 zeta = (2/25) / (2*wn); % damping ratio
28
29 fprintf('Natural frequency (wn) = %.4f rad/s\n', wn);
30 fprintf('Damping ratio (zeta) = %.4f\n', zeta);
31 fprintf('\n');
32
33 % Plot unit impulse response
34 figure;
35 impulse(G1);
36 title('Unit Impulse Response of G(s) = 50/(25s^2+2s+1)');
37 grid on;
38 saveas(gcf, 'results/figures/Task02_Figure_01.png');
39 fprintf('Figure 1 saved: Unit impulse response.\n');
40 fprintf('\n');
41
42 % Get impulse response data
43 [y_imp, t_imp] = impulse(G1);
44 fprintf('Impulse response: max amplitude = %.4f at t = %.4f s\n', ...
45         max(y_imp), t_imp(y_imp == max(y_imp)));
46 fprintf('\n');
47
48 %% Part 2: Exercise 3-7 - Step Response Analysis
49 fprintf('--- Part 2: Step Response Analysis (Exercise 3-7) ---\n');
50 fprintf('\n');
51
52 % System: G(s) = 1/(s(s+1)), unit feedback
53 % Closed-loop: T(s) = G/(1+G) = 1/(s^2 + s + 1)
54 num2 = [1];
55 den2 = [1, 1, 0];
56 G2 = tf(num2, den2); % open loop
57
58 % Closed-loop transfer function
59 T2 = feedback(G2, 1);
60 fprintf('Open-loop: G(s) = 1/(s(s+1))\n');
61 fprintf('Closed-loop: T(s) = 1/(s^2 + s + 1)\n');
62 fprintf('\n');
63
64 % Step response and metrics
65 figure;
66 step(T2);
67 title('Step Response of T(s) = 1/(s^2+s+1)');
68 grid on;
69 saveas(gcf, 'results/figures/Task02_Figure_02.png');
70 fprintf('Figure 2 saved: Step response of closed-loop system.\n');
71
72 % Get step response metrics
73 info = stepinfo(T2);
74 fprintf('Step Response Metrics (G(s)=1/(s(s+1))):\n');

```

```

75 fprintf(' Rise time (tr) = %.4f s\n', info.RiseTime);
76 fprintf(' Peak time (tp) = %.4f s\n', info.PeakTime);
77 fprintf(' Maximum overshoot (Mp) = %.2f %%\n', info.Overshoot);
78 fprintf(' Settling time (ts) = %.4f s\n', info.SettlingTime);
79 fprintf('\n');
80
81 %% Part 3: Analyze effect of K on dynamics
82 fprintf('--- Part 3: Effect of Gain K on Step Response ---\n');
83 fprintf('\n');
84
85 %  $G(s) = K/(s(s+1))$  for  $K = 0.5, 1, 2, 5, 10$ 
86 K_values = [0.5, 1, 2, 5, 10];
87 colors = {'b', 'r', 'g', 'm', 'k'};
88
89 figure;
90 hold on;
91 for i = 1:length(K_values)
92     K = K_values(i);
93     Gk = tf(K, [1, 1, 0]); %  $K/(s(s+1))$ 
94     Tk = feedback(Gk, 1);
95     [y, t] = step(Tk);
96     plot(t, y, colors{i}, 'LineWidth', 1.5, 'DisplayName', sprintf('K
           =%.1f', K));
97 end
98 hold off;
99 xlabel('Time (s)');
100 ylabel('Amplitude');
101 title('Step Response for Different K Values');
102 legend('show', 'Location', 'southeast');
103 grid on;
104 saveas(gcf, 'results/figures/Task02_Figure_03.png');
105 fprintf('Figure 3 saved: Step responses for different K values.\n');
106
107 % Compute and display metrics for each K
108 fprintf('Performance metrics for different K values:\n');
109 fprintf('K\tRise Time\tPeak Time\tOvershoot(%%)\tSettling Time\n');
110 for i = 1:length(K_values)
111     K = K_values(i);
112     Gk = tf(K, [1, 1, 0]);
113     Tk = feedback(Gk, 1);
114     info_k = stepinfo(Tk);
115     fprintf('%.1f\t%.4f\t%.4f\t%.2f\t%.4f\n', ...
116           K, info_k.RiseTime, info_k.PeakTime, info_k.Overshoot, info_k.
           SettlingTime);
117 end
118
119 fprintf('\n');
120 fprintf('Analysis: As K increases, the response becomes faster (
           shorter rise time)\n');

```

```

121 fprintf('but overshoot increases, leading to more oscillatory behavior
    .\n');
122 fprintf('For K=0.5 the system is overdamped, for K=1 it is critically
    damped,\n');
123 fprintf('and for larger K it becomes underdamped with increasing
    overshoot.\n');
124
125 fprintf('\nTask 2 completed.\n');
126 diary off;

```

5.3. Task 03: Bode 图与 Nyquist 图 (task3.m)

```

1 %% task3.m - Experiment 7, Task 3: Bode & Nyquist Plots
2 % Example 4-8: Bode plot with gain/phase margins
3 % Example 4-10: Nyquist plot
4 % Exercise 4-15: Nyquist plots for two systems
5
6 clc;
7 clear;
8 close all;
9
10 % Ensure output directories exist
11 if ~exist('results/logs', 'dir'), mkdir('results/logs'); end
12 if ~exist('results/figures', 'dir'), mkdir('results/figures'); end
13
14 diary('results/logs/Task03_output.log');
15
16 fprintf('==== Task 3: Bode & Nyquist Plots ==== \n');
17 fprintf('\n');
18
19 %% Part 1: Example 4-8 - Bode Plot
20 fprintf('--- Part 1: Bode Plot (Example 4-8) --- \n');
21 fprintf('\n');
22
23 % System:  $G(s) = 50 / (25s^2 + 2s + 1)$ 
24 num = [50];
25 den = [25, 2, 1];
26 G = tf(num, den);
27
28 fprintf('System:  $G(s) = 50 / (25s^2 + 2s + 1)$  \n');
29 fprintf('\n');
30
31 % Bode plot with margin
32 figure;
33 margin(G);
34 title('Bode Plot of  $G(s) = 50/(25s^2+2s+1)$ ');
35 grid on;
36 saveas(gcf, 'results/figures/Task03_Figure_01.png');

```



```

37 fprintf('Figure 1 saved: Bode plot with margins.\n');
38
39 % Get gain and phase margins
40 [Gm, Pm, Wcg, Wcp] = margin(G);
41 fprintf('Gain Margin (Gm) = %.4f (%.2f dB)\n', Gm, 20*log10(Gm));
42 fprintf('Phase Margin (Pm) = %.4f deg\n', Pm);
43 fprintf('Phase crossover frequency (Wcg) = %.4f rad/s\n', Wcg);
44 fprintf('Gain crossover frequency (Wcp) = %.4f rad/s\n', Wcp);
45 fprintf('\n');
46
47 %% Part 2: Example 4-10 - Nyquist Plot
48 fprintf('--- Part 2: Nyquist Plot (Example 4-10) ---\n');
49 fprintf('\n');
50
51 figure;
52 nyquist(G);
53 title('Nyquist Plot of  $G(s) = 50/(25s^2+2s+1)$ ');
54 grid on;
55 saveas(gcf, 'results/figures/Task03_Figure_02.png');
56 fprintf('Figure 2 saved: Nyquist plot.\n');
57
58 % Also show zoomed Nyquist around critical point
59 figure;
60 nyquist(G);
61 title('Nyquist Plot of  $G(s) = 50/(25s^2+2s+1)$  (Zoomed)');
62 axis([-2, 1, -2, 2]);
63 grid on;
64 saveas(gcf, 'results/figures/Task03_Figure_03.png');
65 fprintf('Figure 3 saved: Nyquist plot (zoomed).\n');
66 fprintf('\n');
67
68 %% Part 3: Exercise 4-15 - Nyquist Plots for Two Systems
69 fprintf('--- Part 3: Nyquist Plots (Exercise 4-15) ---\n');
70 fprintf('\n');
71
72 % System (1):  $G(s) = 1/((s+1)(2s+1))$ 
73 num1 = [1];
74 den1 = conv([1, 1], [2, 1]);
75 G1 = tf(num1, den1);
76
77 fprintf('System (1):  $G(s) = 1/((s+1)(2s+1))$ \n');
78 fprintf('\n');
79
80 figure;
81 nyquist(G1);
82 title('Nyquist Plot of  $G(s) = 1/((s+1)(2s+1))$ ');
83 grid on;
84 saveas(gcf, 'results/figures/Task03_Figure_04.png');
85 fprintf('Figure 4 saved: Nyquist plot for System (1).\n');

```

```

86
87 % System (2):  $G(s) = 1/(s^2(s+1)(2s+1))$ 
88 num2 = [1];
89 den2 = conv([1, 0, 0], conv([1, 1], [2, 1]));
90 G2 = tf(num2, den2);
91
92 fprintf('System (2):  $G(s) = 1/(s^2(s+1)(2s+1))\n$ ');
93 fprintf('\n');
94
95 figure;
96 nyquist(G2);
97 title('Nyquist Plot of  $G(s) = 1/(s^2(s+1)(2s+1))$ ');
98 grid on;
99 saveas(gcf, 'results/figures/Task03_Figure_05.png');
100 fprintf('Figure 5 saved: Nyquist plot for System (2).\n');
101
102 % Compare the two Nyquist plots
103 figure;
104 subplot(1, 2, 1);
105 nyquist(G1);
106 title('System (1):  $G(s)=1/((s+1)(2s+1))$ ');
107 grid on;
108 subplot(1, 2, 2);
109 nyquist(G2);
110 title('System (2):  $G(s)=1/(s^2(s+1)(2s+1))$ ');
111 grid on;
112 saveas(gcf, 'results/figures/Task03_Figure_06.png');
113 fprintf('Figure 6 saved: Comparison of Nyquist plots.\n');
114
115 fprintf('\nTask 3 completed.\n');
116 diary off;

```

5.4. Task 04: 稳定裕度分析 (task4.m)

```

1 %% task4.m - Experiment 7, Task 4: Stability Margins
2 % Example 5-21: Nyquist and Bode plots, compute phase and gain margins
3 % Exercise 5-22: Bode diagram and stability analysis
4
5 clc;
6 clear;
7 close all;
8
9 % Ensure output directories exist
10 if ~exist('results/logs', 'dir'), mkdir('results/logs'); end
11 if ~exist('results/figures', 'dir'), mkdir('results/figures'); end
12
13 diary('results/logs/Task04_output.log');
14

```

```

15 fprintf('==== Task 4: Stability Margins ==== \n');
16 fprintf('\n');
17
18 %% Part 1: Example 5-21 - Stability Margins Analysis
19 fprintf('--- Part 1: Stability Margins (Example 5-21) --- \n');
20 fprintf('\n');
21
22 % System:  $G(s) = 40/(s(0.1s+1)(0.05s+1))$ 
23 num1 = [40];
24 den1 = conv([1, 0], conv([0.1, 1], [0.05, 1]));
25 G1 = tf(num1, den1);
26
27 fprintf('System:  $G(s) = 40/(s(0.1s+1)(0.05s+1))$  \n');
28 disp(G1);
29 fprintf('\n');
30
31 % Bode plot with margins
32 figure;
33 margin(G1);
34 title('Bode Plot of  $G(s) = 40/(s(0.1s+1)(0.05s+1))$ ');
35 grid on;
36 saveas(gcf, 'results/figures/Task04_Figure_01.png');
37 fprintf('Figure 1 saved: Bode plot with margins. \n');
38
39 % Get margin values
40 [Gm1, Pm1, Wcg1, Wcp1] = margin(G1);
41 fprintf('Gain Margin (Gm) = %.4f (%.2f dB) \n', Gm1, 20*log10(Gm1));
42 fprintf('Phase Margin (Pm) = %.4f deg \n', Pm1);
43 fprintf('Phase crossover frequency (Wcg) = %.4f rad/s \n', Wcg1);
44 fprintf('Gain crossover frequency (Wcp) = %.4f rad/s \n', Wcp1);
45 fprintf('\n');
46
47 % Nyquist plot
48 figure;
49 nyquist(G1);
50 title('Nyquist Plot of  $G(s) = 40/(s(0.1s+1)(0.05s+1))$ ');
51 grid on;
52 saveas(gcf, 'results/figures/Task04_Figure_02.png');
53 fprintf('Figure 2 saved: Nyquist plot. \n');
54
55 % Zoomed Nyquist around critical point
56 figure;
57 nyquist(G1);
58 title('Nyquist Plot (Zoomed)');
59 axis([-3, 1, -3, 3]);
60 grid on;
61 saveas(gcf, 'results/figures/Task04_Figure_03.png');
62 fprintf('Figure 3 saved: Nyquist plot (zoomed). \n');
63

```

```

64 % Check stability based on margins
65 if Pm1 > 0 && Gm1 > 1
66     fprintf('System is stable (positive gain and phase margins).\n');
67 else
68     fprintf('System is unstable (negative gain or phase margin).\n');
69 end
70 fprintf('\n');
71
72 %% Part 2: Exercise 5-22 - Bode and Stability Analysis
73 fprintf('--- Part 2: Stability Analysis (Exercise 5-22) ---\n');
74 fprintf('\n');
75
76 % System:  $G(s) = 10/(s(s+1)(s+10))$ 
77 num2 = [10];
78 den2 = conv([1, 0], conv([1, 1], [1, 10]));
79 G2 = tf(num2, den2);
80
81 fprintf('System:  $G(s) = 10/(s(s+1)(s+10))$ \n');
82 disp(G2);
83 fprintf('\n');
84
85 % Bode plot
86 figure;
87 margin(G2);
88 title('Bode Plot of  $G(s) = 10/(s(s+1)(s+10))$ ');
89 grid on;
90 saveas(gcf, 'results/figures/Task04_Figure_04.png');
91 fprintf('Figure 4 saved: Bode plot with margins.\n');
92
93 % Get margin values
94 [Gm2, Pm2, Wcg2, Wcp2] = margin(G2);
95 fprintf('Gain Margin (Gm) = %.4f (%.2f dB)\n', Gm2, 20*log10(Gm2));
96 fprintf('Phase Margin (Pm) = %.4f deg\n', Pm2);
97 fprintf('Phase crossover frequency (Wcg) = %.4f rad/s\n', Wcg2);
98 fprintf('Gain crossover frequency (Wcp) = %.4f rad/s\n', Wcp2);
99 fprintf('\n');
100
101 % Nyquist plot
102 figure;
103 nyquist(G2);
104 title('Nyquist Plot of  $G(s) = 10/(s(s+1)(s+10))$ ');
105 grid on;
106 saveas(gcf, 'results/figures/Task04_Figure_05.png');
107 fprintf('Figure 5 saved: Nyquist plot.\n');
108
109 % Zoomed Nyquist around critical point
110 figure;
111 nyquist(G2);
112 title('Nyquist Plot of  $G(s) = 10/(s(s+1)(s+10))$  (Zoomed)');

```

```

113 axis([-2, 1, -2, 2]);
114 grid on;
115 saveas(gcf, 'results/figures/Task04_Figure_06.png');
116 fprintf('Figure 6 saved: Nyquist plot (zoomed).\n');
117
118 % Stability assessment using Nyquist criterion
119 fprintf('Stability Assessment:\n');
120 fprintf(' Based on Bode plot: ');
121 if Pm2 > 0 && Gm2 > 1
122     fprintf('System appears stable (positive margins).\n');
123 elseif Pm2 > 0 && Gm2 < 1
124     fprintf('System may be unstable (gain margin < 1).\n');
125 else
126     fprintf('System may be unstable (negative phase margin).\n');
127 end
128
129 % Check closed-loop poles for definitive answer
130 T2 = feedback(G2, 1);
131 poles_cl = pole(T2);
132 fprintf(' Closed-loop poles:\n');
133 fprintf(' %s\n', sprintf('%.4f %+.4fi ', real(poles_cl), imag(
    poles_cl)));
134 if all(real(poles_cl) < 0)
135     fprintf(' Closed-loop system is STABLE (all poles in LHP).\n');
136 else
137     fprintf(' Closed-loop system is UNSTABLE (right-half-plane poles)
        .\n');
138 end
139 fprintf('\n');
140
141 % Step response to verify
142 figure;
143 step(T2);
144 title('Step Response of Closed-Loop System');
145 grid on;
146 saveas(gcf, 'results/figures/Task04_Figure_07.png');
147 fprintf('Figure 7 saved: Step response.\n');
148
149 fprintf('\nTask 4 completed.\n');
150 diary off;

```

5.5. Task 05: PID 控制器设计 (task5.m)

```

1 %% task5.m - Experiment 7, Task 5: PID Controller Design
2 % Example 7-6: Phase margins before and after correction
3 % Exercise 7-20: PID controller design using pole placement
4
5 clc;

```

```

6 clear;
7 close all;
8
9 % Ensure output directories exist
10 if ~exist('results/logs', 'dir'), mkdir('results/logs'); end
11 if ~exist('results/figures', 'dir'), mkdir('results/figures'); end
12
13 diary('results/logs/Task05_output.log');
14
15 fprintf('==== Task 5: PID Controller Design ==== \n');
16 fprintf('\n');
17
18 %% Part 1: Example 7-6 - Phase Margin Comparison Before/After
    Correction
19 fprintf('--- Part 1: Phase Margin Analysis (Example 7-6) --- \n');
20 fprintf('\n');
21
22 % System before correction:  $G_1(s) = 100/(s(0.04s+1)(0.01s+1))$ 
23 num1 = [100];
24 den1 = conv([1, 0], conv([0.04, 1], [0.01, 1]));
25 G1 = tf(num1, den1);
26
27 fprintf('Before correction:  $G_1(s) = 100/(s(0.04s+1)(0.01s+1))$  \n');
28 disp(G1);
29 fprintf('\n');
30
31 % Bode plot with margins before correction
32 figure;
33 margin(G1);
34 title('Bode Plot - Before Correction');
35 grid on;
36 saveas(gcf, 'results/figures/Task05_Figure_01.png');
37 fprintf('Figure 1 saved: Bode plot before correction. \n');
38
39 [Gm1, Pm1, Wcg1, Wcp1] = margin(G1);
40 fprintf('Before Correction: \n');
41 fprintf('  Gain Margin = %.4f (%.2f dB) \n', Gm1, 20*log10(Gm1));
42 fprintf('  Phase Margin = %.4f deg \n', Pm1);
43 fprintf('  Phase crossover freq = %.4f rad/s \n', Wcg1);
44 fprintf('  Gain crossover freq = %.4f rad/s \n', Wcp1);
45 if Pm1 > 0 && Gm1 > 1
46     fprintf('  System is stable. \n');
47 else
48     fprintf('  System is unstable. \n');
49 end
50 fprintf('\n');
51
52 % System after correction:  $G_2(s) = 100(0.5s+1)/(s(5s+1)(0.04s+1)(0.01s+1))$ 

```

```

53 num2 = 100 * [0.5, 1];
54 den2 = conv([1, 0], conv([5, 1], conv([0.04, 1], [0.01, 1])));
55 G2 = tf(num2, den2);
56
57 fprintf('After correction: G2(s) = 100(0.5s+1)/(s(5s+1)(0.04s+1)(0.01s
    +1))\n');
58 disp(G2);
59 fprintf('\n');
60
61 % Bode plot with margins after correction
62 figure;
63 margin(G2);
64 title('Bode Plot - After Correction');
65 grid on;
66 saveas(gcf, 'results/figures/Task05_Figure_02.png');
67 fprintf('Figure 2 saved: Bode plot after correction.\n');
68
69 [Gm2, Pm2, Wcg2, Wcp2] = margin(G2);
70 fprintf('After Correction:\n');
71 fprintf(' Gain Margin = %.4f (%.2f dB)\n', Gm2, 20*log10(Gm2));
72 fprintf(' Phase Margin = %.4f deg\n', Pm2);
73 fprintf(' Phase crossover freq = %.4f rad/s\n', Wcg2);
74 fprintf(' Gain crossover freq = %.4f rad/s\n', Wcp2);
75 if Pm2 > 0 && Gm2 > 1
76     fprintf(' System is stable.\n');
77 else
78     fprintf(' System is unstable.\n');
79 end
80 fprintf('\n');
81
82 % Comparison
83 fprintf('Comparison:\n');
84 fprintf(' Phase margin improved from %.2f to %.2f degrees.\n', Pm1,
    Pm2);
85 fprintf(' Gain margin improved from %.2f dB to %.2f dB.\n', 20*log10(
    Gm1), 20*log10(Gm2));
86 fprintf('\n');
87
88 % Step response comparison
89 figure;
90 [y1, t1] = step(feedback(G1, 1));
91 [y2, t2] = step(feedback(G2, 1));
92 plot(t1, y1, 'b-', 'LineWidth', 1.5, 'DisplayName', 'Before correction
    ');
93 hold on;
94 plot(t2, y2, 'r-', 'LineWidth', 1.5, 'DisplayName', 'After correction'
    );
95 hold off;
96 xlabel('Time (s)');

```

```

97 ylabel('Amplitude');
98 title('Step Response Comparison: Before vs After Correction');
99 legend('show');
100 grid on;
101 saveas(gcf, 'results/figures/Task05_Figure_03.png');
102 fprintf('Figure 3 saved: Step response comparison.\n');
103 fprintf('\n');
104
105 %% Part 2: Exercise 7-20 - PID Controller Design via Pole Placement
106 fprintf('--- Part 2: PID Controller Design (Exercise 7-20) ---\n');
107 fprintf('\n');
108
109 % Plant:  $G_0(s) = 100/(s(10s+1))$ 
110 num0 = [100];
111 den0 = [10, 1, 0];
112 G0 = tf(num0, den0);
113
114 fprintf('Plant:  $G_0(s) = 100/(s(10s+1))$ \n');
115 disp(G0);
116 fprintf('\n');
117
118 % Desired closed-loop poles:  $-2+j1$ ,  $-2-j1$ ,  $-5$ 
119 desired_poles = [-2+1i, -2-1i, -5];
120
121 fprintf('Desired closed-loop poles:\n');
122 fprintf('  p1 = -2 + j1\n');
123 fprintf('  p2 = -2 - j1\n');
124 fprintf('  p3 = -5\n');
125 fprintf('\n');
126
127 % PID gains designed via coefficient matching
128 Kp = 2.5;
129 Ki = 2.5;
130 Kd = 0.89;
131
132 fprintf('PID Controller Parameters (designed):\n');
133 fprintf('  Kp (proportional) = %.4f\n', Kp);
134 fprintf('  Ki (integral) = %.4f\n', Ki);
135 fprintf('  Kd (derivative) = %.4f\n', Kd);
136 fprintf('\n');
137
138 % Build PID controller
139 s = tf('s');
140 C = Kp + Ki/s + Kd*s;
141 fprintf('PID Controller:  $C(s) = %.4f + %.4f/s + %.4f*s$ \n', Kp, Ki, Kd)
142 ;
143 fprintf('\n');
144 % Closed-loop system

```



```

145 G_cl = feedback(C * G0, 1);
146 fprintf('Closed-loop transfer function:\n');
147 disp(G_cl);
148 fprintf('\n');
149
150 % Verify pole placement
151 poles_actual = pole(G_cl);
152 fprintf('Actual closed-loop poles:\n');
153 for i = 1:length(poles_actual)
154     fprintf('  p%d = %.4f %.4fi\n', i, real(poles_actual(i)), imag(
        poles_actual(i)));
155 end
156 fprintf('\n');
157
158 % Check if poles match desired
159 fprintf('Pole placement verification:\n');
160 fprintf('  Desired: -2+j1, -2-j1, -5\n');
161 fprintf('  Achieved: ');
162 for i = 1:length(poles_actual)
163     fprintf(' (%.4f%.4fi) ', real(poles_actual(i)), imag(poles_actual(
        i)));
164 end
165 fprintf('\n');
166 fprintf('\n');
167
168 % Plot step response
169 figure;
170 step(G_cl);
171 title('Step Response with PID Controller (Pole Placement)');
172 grid on;
173 saveas(gcf, 'results/figures/Task05_Figure_04.png');
174 fprintf('Figure 4 saved: Step response with PID controller.\n');
175
176 % Step response metrics
177 try
178     info_pid = stepinfo(G_cl);
179     fprintf('Step Response Metrics with PID:\n');
180     fprintf('  Rise time = %.4f s\n', info_pid.RiseTime);
181     fprintf('  Peak time = %.4f s\n', info_pid.PeakTime);
182     fprintf('  Overshoot = %.2f %%\n', info_pid.Overshoot);
183     fprintf('  Settling time = %.4f s\n', info_pid.SettlingTime);
184     fprintf('  Final value = %.4f\n', dcgain(G_cl));
185 catch
186     fprintf('Stepinfo computation failed.\n');
187 end
188 fprintf('\n');
189
190 % Bode plot of the compensated system
191 figure;

```

```

192 margin(C * G0);
193 title('Bode Plot of Compensated System (C(s)*G0(s))');
194 grid on;
195 saveas(gcf, 'results/figures/Task05_Figure_05.png');
196 fprintf('Figure 5 saved: Bode plot of compensated system.\n');
197
198 fprintf('\nTask 5 completed.\n');
199 diary off;

```

5.6. Task 06: 根轨迹分析 (task6.m)

```

1 %% task6.m - Experiment 7, Task 6: Root Locus Analysis
2 % Example 8-12: Root locus of  $G(s) = K/(s(s+2)(s+5))$ 
3 % Exercise 8-8: Root locus with parameter variation ( $Kh$ )
4
5 clc;
6 clear;
7 close all;
8
9 % Ensure output directories exist
10 if ~exist('results/logs', 'dir'), mkdir('results/logs'); end
11 if ~exist('results/figures', 'dir'), mkdir('results/figures'); end
12
13 diary('results/logs/Task06_output.log');
14
15 fprintf('==== Task 6: Root Locus Analysis ==== \n');
16 fprintf('\n');
17
18 %% Part 1: Example 8-12 - Root Locus
19 fprintf('--- Part 1: Root Locus (Example 8-12) --- \n');
20 fprintf('\n');
21
22 % System:  $G(s) = K/(s(s+2)(s+5))$ 
23 num1 = [1];
24 den1 = conv([1, 0], conv([1, 2], [1, 5]));
25 G1 = tf(num1, den1);
26
27 fprintf('System:  $G(s) = K/(s(s+2)(s+5))$  \n');
28 disp(G1);
29 fprintf('\n');
30
31 % Plot root locus
32 figure;
33 rlocus(G1);
34 title('Root Locus of  $G(s) = K/(s(s+2)(s+5))$ ');
35 grid on;
36 saveas(gcf, 'results/figures/Task06_Figure_01.png');
37 fprintf('Figure 1 saved: Root locus.\n');

```

```

38
39 % Find gain at selected point on root locus
40 s_test = -1 + 1i; % Test point
41 K_val_at_point = -s_test*(s_test+2)*(s_test+5);
42 fprintf('At test point s = %.4f %+.4fi:\n', real(s_test), imag(s_test)
    );
43 fprintf(' Corresponding gain K = %.4f\n', abs(K_val_at_point));
44 fprintf('\n');
45
46 % Verify this point relative to root locus
47 G1_s = polyval(num1, s_test) / polyval(den1, s_test);
48 angle_G1 = angle(G1_s) * 180/pi;
49 fprintf('Angle of G(s) at test point: %.2f deg\n', angle_G1);
50 fprintf('Angle condition for root locus: %+.2f deg (target: +/-180 deg
    )\n', ...
51         mod(angle_G1 + 180, 360) - 180);
52 fprintf('\n');
53
54 % Find closed-loop poles for this K
55 fprintf('Closed-loop poles for K = %.4f:\n', abs(K_val_at_point));
56 G1_cl = feedback(abs(K_val_at_point) * G1, 1);
57 p1 = pole(G1_cl);
58 for i = 1:length(p1)
59     fprintf(' p%d = %.4f %+.4fi\n', i, real(p1(i)), imag(p1(i)));
60 end
61 fprintf('\n');
62
63 % Find gain at breakaway point
64 s_break = (-14 + sqrt(76))/6;
65 K_break = -(s_break^3 + 7*s_break^2 + 10*s_break);
66 fprintf('Breakaway point: s = %.4f\n', s_break);
67 fprintf('Gain at breakaway: K = %.4f\n', K_break);
68 fprintf('\n');
69
70 %% Part 2: Exercise 8-8 - Root Locus with Parameter Variation
71 fprintf('--- Part 2: Root Locus with Parameter Variation (Exercise
    8-8) ---\n');
72 fprintf('\n');
73
74 %% Part (1): Kh = 0.5, K from 0 to inf
75 fprintf('Part (1): Kh = 0.5, vary K from 0 to inf\n');
76 fprintf('\n');
77
78 Kh = 0.5;
79 % Open-loop: G(s)H(s) = K * (1+Kh*s)/(s(s+1))
80 num_ol = [Kh, 1]; % 0.5*s + 1
81 den_ol = [1, 1, 0]; % s^2 + s
82 G_ol1 = tf(num_ol, den_ol);
83

```

```

84 fprintf('Equivalent open-loop:  $G(s)H(s) = K(1+0.1s)/(s(s+1))$ \n', Kh
    );
85 disp(G_ol1);
86 fprintf('\n');
87
88 figure;
89 rlocus(G_ol1);
90 title(sprintf('Root Locus for Kh=%.1f, K from 0 to inf', Kh));
91 grid on;
92 saveas(gcf, 'results/figures/Task06_Figure_02.png');
93 fprintf('Figure 2 saved: Root locus for Kh=0.5, varying K.\n');
94
95 %% Part (2): Kh = 0.5, K = 10, find closed-loop poles and zeta
96 fprintf('Part (2): Kh = 0.5, K = 10\n');
97 fprintf('\n');
98
99 K_val = 10;
100 Kh_val = 0.5;
101
102 fprintf('Characteristic equation:  $s^2 + (1+K*Kh)s + K = 0$ \n');
103 fprintf('With K=%d, Kh=%.1f:  $s^2 + 0.1s + %d = 0$ \n', K_val, Kh_val,
    1+K_val*Kh_val, K_val);
104
105 % Find poles
106 poles_part2 = roots([1, 1+K_val*Kh_val, K_val]);
107 fprintf('Closed-loop poles:\n');
108 for i = 1:length(poles_part2)
109     fprintf('  s%d = %.4f %.4fi\n', i, real(poles_part2(i)), imag(
        poles_part2(i)));
110 end
111 fprintf('\n');
112
113 % Calculate damping ratio
114 wn_part2 = sqrt(K_val);
115 zeta_part2 = (1 + K_val*Kh_val) / (2*wn_part2);
116 fprintf('Natural frequency: wn = %.4f rad/s\n', wn_part2);
117 fprintf('Damping ratio: zeta = %.4f\n', zeta_part2);
118 fprintf('\n');
119
120 %% Part (3): K = 1, Kh from 0 to inf (parameter root locus)
121 fprintf('Part (3): K = 1, vary Kh from 0 to inf (parameter root locus)
    \n');
122 fprintf('\n');
123
124 % Equivalent open-loop for Kh:  $G_{eq}(s) = s/(s^2+s+1)$ 
125 num_eq = [1, 0]; % s
126 den_eq = [1, 1, 1]; %  $s^2 + s + 1$ 
127 Geq = tf(num_eq, den_eq);
128

```

```

129 fprintf('Equivalent open-loop for parameter Kh: Geq(s) = s/(s^2+s+1)\n
    ');
130 disp(Geq);
131 fprintf('\n');
132
133 figure;
134 rlocus(Geq);
135 title('Parameter Root Locus for Kh (K=1, vary Kh from 0 to inf)');
136 grid on;
137 saveas(gcf, 'results/figures/Task06_Figure_03.png');
138 fprintf('Figure 3 saved: Parameter root locus for Kh.\n');
139
140 %% Part (4): K = 1, Kh = 0, 0.5, 4 - Step response
141 fprintf('Part (4): Step response metrics for different Kh values\n');
142 fprintf('\n');
143
144 K_const = 1;
145 Kh_list = [0, 0.5, 4];
146 colors = {'b', 'r', 'g'};
147
148 figure;
149 hold on;
150 for i = 1:length(Kh_list)
151     Kh_i = Kh_list(i);
152     num_cl = K_const;
153     den_cl = [1, 1+K_const*Kh_i, K_const];
154     T_i = tf(num_cl, den_cl);
155
156     [y_i, t_i] = step(T_i);
157     plot(t_i, y_i, colors{i}, 'LineWidth', 1.5, ...
158         'DisplayName', sprintf('Kh=%.1f', Kh_i));
159 end
160 hold off;
161 xlabel('Time (s)');
162 ylabel('Amplitude');
163 title('Step Response for Different Kh Values (K=1)');
164 legend('show', 'Location', 'southeast');
165 grid on;
166 saveas(gcf, 'results/figures/Task06_Figure_04.png');
167 fprintf('Figure 4 saved: Step responses for different Kh values.\n');
168
169 % Compute and display metrics
170 fprintf('Step Response Metrics (K=1):\n');
171 fprintf('Kh\tOvershoot Mp(%%)\tSettling Time ts(s)\tPeak Time(s)\tRise
    Time(s)\n');
172
173 for i = 1:length(Kh_list)
174     Kh_i = Kh_list(i);
175     num_cl = K_const;

```

```

176     den_cl = [1, 1+K_const*Kh_i, K_const];
177     T_i = tf(num_cl, den_cl);
178
179     try
180         info_i = stepinfo(T_i);
181         fprintf('%.1f\t%.2f\t\t%.4f\t\t%.4f\t\t%.4f\n', ...
182             Kh_i, info_i.Overshoot, info_i.SettlingTime, ...
183             info_i.PeakTime, info_i.RiseTime);
184     catch
185         fprintf('%.1f\tstepinfo failed for this Kh value\n', Kh_i);
186     end
187 end
188 fprintf('\n');
189
190 % Discussion of Kh effect
191 fprintf('Discussion on Kh effect on system dynamics:\n');
192 fprintf(' - When Kh=0 (no derivative feedback), the system is
    underdamped\n');
193 fprintf('     with overshoot Mp = %.2f%%. \n', 16.29);
194 fprintf(' - As Kh increases (Kh=%.1f), damping increases
    significantly,\n', Kh_list(2));
195 fprintf('     reducing overshoot to Mp = %.2f%%. \n', 2.84);
196 fprintf(' - For Kh=%.1f, the system is heavily damped (or overdamped)
    \n', Kh_list(3));
197 fprintf('     with Mp = %.2f%% and longer settling time.\n', 0.00);
198 fprintf(' - Conclusion: Increasing Kh increases the damping ratio,\n'
    );
199 fprintf('     reduces overshoot, but may increase settling time for
    large Kh.\n');
200
201 fprintf('\nTask 6 completed.\n');
202 diary off;

```